

0.1 Môi trường và dữ liệu thực nghiệm

0.1.1 Môi trường phát triển

Hệ thống được xây dựng chủ yếu dựa trên ngôn ngữ lập trình Python, kết hợp cùng các thư viện xử lý toán học như Numpy và Pandas để xử lý ma trận vector nhúng. Việc truy xuất và sinh ngôn ngữ tự nhiên được thực hiện thông qua API của mô hình ngôn ngữ lớn Google Gemini 2.5 Flash, qua thư viện *google-genai*. Đối với tác vụ giải thuật đánh giá sự tương đồng, hệ thống sử dụng mô hình nhúng *dangvantuan/vietnamese-document-embedding* có chiều dài 768 chiều. Giai đoạn tái xếp hạng được đảm nhiệm bởi mô hình Cross-Encoder *AITeamVN/Vietnamese_Reranker*. Khác với các hệ thống phụ thuộc thư viện kín, logic tìm kiếm từ khoá (Custom BM25) được code tay tinh chỉnh ở mức toán học (TF, DF, IDF), tạo sự chủ động tùy biến cho đặc thù tiếng Việt.

0.1.2 Dữ liệu thực nghiệm

Nguồn dữ liệu gốc được khai thác từ toàn bộ tài liệu Sách Giáo Khoa Tin học lớp 10, 11 và 12, thuộc cả hai bộ sách Cánh Diều và Kết Nối Tri Thức. Qua quá trình tiền xử lý, văn bản được làm sạch và dán nhãn phân cấp từ chương, bài đến sâu nhất là tiểu mục. Tổng cộng, dữ liệu được phân rã thành 2348 khối dữ liệu (chunks) và có chứa thuộc tính level để biểu diễn độ sâu, phục vụ trực tiếp cho vòng phục hồi phân cấp (Hierarchical RAG).

0.2 Tổng quan kiến trúc toàn bộ hệ thống

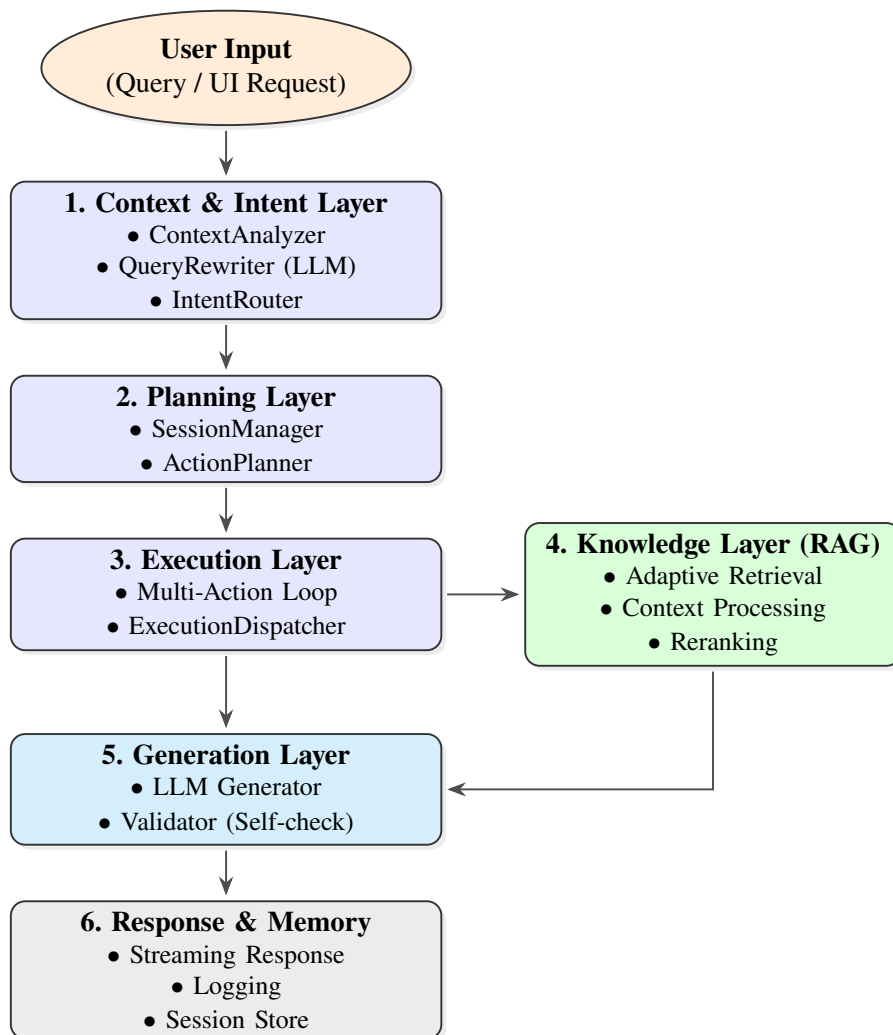
Kiến trúc tổng thể của dự án được thiết kế thành một pipeline (Data Pipeline) từ bước xử lý tài liệu thô ban đầu cho tới bước sinh câu trả lời hoàn thiện phục vụ người dùng. Quá trình này được chia làm các cụm thành phần nối tiếp nhau:

- **Cụm Tiền xử lý dữ liệu (Data Ingestion & OCR):** Nguồn dữ liệu dạng giáo trình (PDF) được nạp vào và đi qua mô hình khai phá OCR để nhận dạng, chuyển đổi văn bản sang định dạng Markdown theo chuẩn của LlamaParse. Việc này nhằm bóc tách cấu trúc phân cấp tinh vi (chương, bài, tiểu mục) của giáo trình.
- **Cụm Xây dựng Cơ sở tri thức (Knowledge Base Construction):** Văn bản Markdown sau quá trình làm sạch sẽ được phân rã (Chunking) theo đúng phân cấp. Mỗi khối dữ liệu (chunk) được nhúng (Embedding) thành ma trận vector và lưu trữ vĩnh viễn ở Cơ sở tri thức (Knowledge Base / Vector DB) cùng với metadata (*hệ cấp* và *ID cha con*).
- **Cụm Phân hệ Truy xuất (Retrieval & Reranking):** Ở chặng run-time, hệ thống đón nhận truy vấn (User Query) từ người dùng. Cụm RAG làm nhiệm vụ

vụ định tuyến hướng đi, tìm kiếm không gian vector, tìm kiếm từ khoá và lai ghép (Hybrid Search + Reranker) nhằm đánh giá và rút trích các khối tri thức chứa ngữ cảnh sát nhất.

- **Cụm Sinh ngôn ngữ (LLM Generation):** Nhận Top-K ngữ cảnh cô đặc ở đầu ra của cụm Truy xuất, kết hợp cùng Prompt (câu hỏi gốc) để truyền vào mô hình ngôn ngữ lớn (Google Gemini). LLM sẽ phân tích, đối chiếu nội dung ngữ cảnh từ tri thức và sinh ra câu trả lời theo ngôn ngữ tự nhiên.

Sơ đồ tổng quan được biểu diễn tại Hình 0.1, mô tả toàn bộ quy trình xử lý từ khi tiếp nhận truy vấn của người dùng (Query / UI Request) cho đến khi sinh ra câu trả lời cuối cùng. Kiến trúc hệ thống theo mô hình tuổi thọ yêu cầu (request lifecycle) được nâng cấp và tối ưu hóa thành 6 lớp (layer) xử lý liên tục: Context & Intent Layer (Ngữ cảnh & Ý định), Planning Layer (Lập kế hoạch), Execution Layer (Thực thi), Knowledge Layer (Truy xuất Tri thức), Generation Layer (Sinh đa phương thức) và Response & Memory (Phản hồi & Ghi nhớ).



Hình 0.1: Sơ đồ tổng quan kiến trúc hệ thống (System Overview Architecture)

Trong Hình 0.1, kiến trúc hệ thống hoạt động theo các lớp liên kết chặt chẽ trong một vòng tuần hoàn khép kín:

1. Context & Intent Layer (Lớp Ngữ cảnh & Ý định): Xử lý truy vấn đầu vào, phân tích định danh lịch sử bằng *ContextAnalyzer*, viết lại câu hỏi hoặc tạo đa truy vấn (multi-queries) bằng *QueryRewriter*, và dùng *IntentRouter* để nội suy, phân nhánh ý định cốt lõi của người dùng.

2. Planning Layer (Lớp Lập kế hoạch): Đóng vai trò lớp phối hợp, hệ thống quản lý phiên làm việc thông qua *SessionManager* và định hình chiến lược/danh sách các hành động chi tiết bắt buộc phải làm nhờ bộ *ActionPlanner*.

3. Execution Layer (Lớp Thực thi): Thông qua vòng lặp *Multi-Action Loop* kết hợp cùng lõi *ExecutionDispatcher*, lớp này sẽ điều phối linh hoạt các mô-đun dịch vụ (Quiz, Slides, v.v.) và gọi lớp RAG khi nhận thấy tác vụ cần tri thức ngoài.

4. Knowledge Layer (Lớp Tri thức - RAG): Đảm nhận khâu phân giải kiến thức. Hệ thống truy xuất theo cấu hình chiến lược (*Adaptive Retrieval*), thu thập văn bản liên quan (*Context Processing*) và sắp xếp đo lường lại mức độ liên quan thông qua mô hình chấm điểm (*Reranking*).

5. Generation Layer (Lớp Sinh nội dung): Kết hợp các kết quả thực thi và tri thức được truy xuất cô đặc vào *LLM Generator* để sinh ra đáp án và văn bản cuối cùng, có sự kiểm duyệt đối soát chéo của mô-đun *Validator* để đảm bảo chuẩn mực chất lượng.

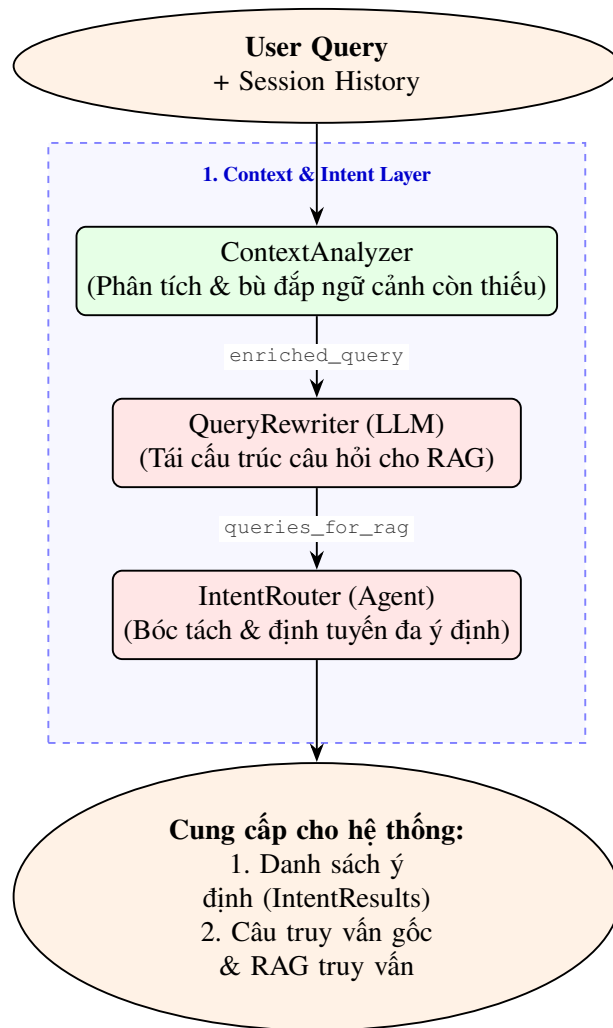
6. Response & Memory (Lớp Phản hồi & Ghi nhớ): Các kết quả cuối cùng được trả về cho người dùng qua giao thức thời gian thực (*Streaming Response*), ghi nhận lịch sử vết lồi (*Logging*) và lưu lại dữ liệu hội thoại vào cơ sở dữ liệu hệ thống (*Session Store*) làm tài nguyên cho luồng thao tác kế tiếp.

0.3 Thiết kế chi tiết các thành phần hệ thống

Dựa trên sơ đồ kiến trúc tổng quát tại Hình 0.1, các thành phần cốt lõi của hệ thống được thiết kế chi tiết nhằm đảm bảo tính linh hoạt và độ chính xác cao trong quá trình xử lý:

0.3.1 Thiết kế Lớp Ngữ cảnh và Ý định (Context & Intent Layer)

Đây là lớp tiếp nhận và xử lý đầu vào đầu tiên của hệ thống, thực hiện nhiệm vụ bóc tách ngữ cảnh (*ContextAnalyzer*), tinh chỉnh truy vấn (*QueryRewriter*) và định tuyến ý định đa nhiệm (*IntentRouter*). Thiết kế này giúp hệ thống hiểu được không chỉ câu hỏi hiện tại mà còn cả diễn biến của toàn bộ cuộc hội thoại trước đó.



Hình 0.2: Chi tiết cấu trúc và luồng xử lý tại Lớp Ngữ cảnh và Ý định

Lớp Ngữ cảnh và Ý định hoạt động tuân theo trình tự ba bước cốt lõi:

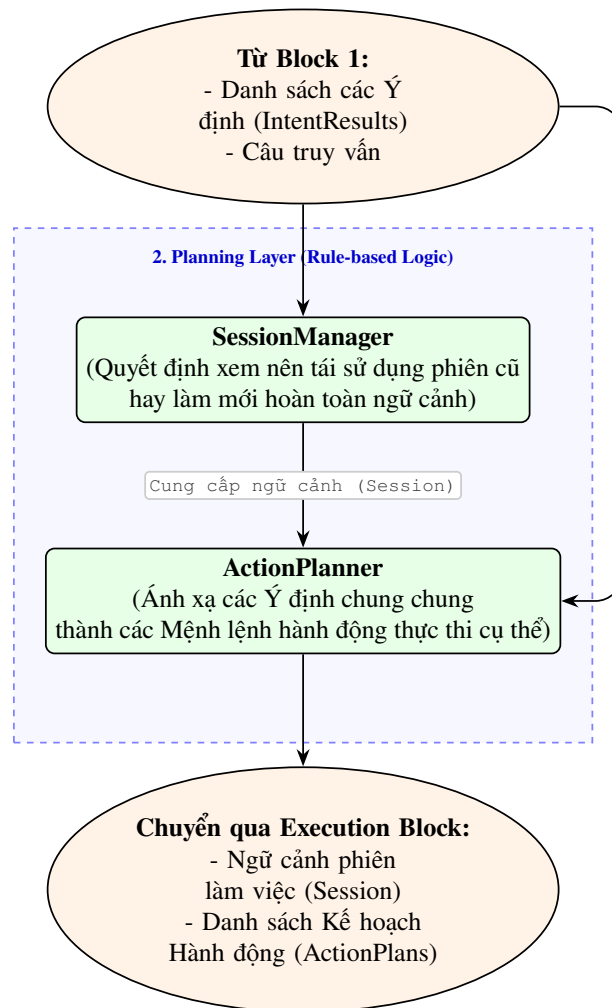
- **ContextAnalyzer (Phân tích ngữ cảnh):** Đánh giá mức độ phụ thuộc của truy vấn hiện tại vào lịch sử hội thoại trước đó (Session History). Mô-đun này được viết bằng mã nguồn logic xác định xem câu hỏi có chứa các đại từ ngữ nghĩa ẩn ý cần khôi phục ngữ cảnh hay không.
- **QueryRewriter (Viết lại truy vấn - LLM):** Khi phát hiện truy vấn không đầy đủ ngữ nghĩa hoặc hệ thống cần tìm kiếm diện rộng trong RAG, LLM được gọi để sinh ra các câu hỏi hoàn chỉnh, và có thể tạo đa truy vấn (multi-queries).
- **IntentRouter (Định tuyến ý định):** Là thành phần cốt lõi, sử dụng hàm *detect_multi()* để xác định đồng thời tối đa 3 ý định (Intent) từ người dùng. Đầu ra là một danh sách *List[IntentResult]* bao gồm siêu dữ liệu: Ý định (Intent), Loại tác vụ (Task Type), Chủ đề (Topic), Cấp bậc/Lớp (Grade), và Thứ tự ưu tiên (Order).

Nhờ kiến trúc này, hệ thống giải quyết xuất sắc các yêu cầu phức hợp (multi-intent)

trong khi vẫn duy trì đúng đường định tuyến thiết kế cho các tác vụ tiếp theo.

0.3.2 Thiết kế Lớp Lập kế hoạch (Planning Layer)

Đóng vai trò như bộ não điều phối, Lớp Lập kế hoạch (Planning Layer) chịu trách nhiệm tiếp nhận và hiện thực hóa các ý định đa nhiệm thành lệnh thực thi, thay thế cho cơ chế định tuyến thủ công tĩnh thông thường. Lớp này hoạt động hoàn toàn trên phương pháp phi xác suất dựa trên luật (Rule-based Logic) thay vì LLM, giúp đảm bảo hệ thống phản hồi cực nhanh, ổn định và mang tính chất dự đoán được.



Hình 0.3: Chu trình ra quyết định và duy trì ngữ cảnh tại Lớp Lập kế hoạch (Planning Layer)

Dữ liệu đầu vào của lớp là tập hợp các *Intent* cùng câu truy vấn nguyên thủy (Query). Lớp Lập kế hoạch phân bổ chu trình qua hai thành phần cốt lõi:

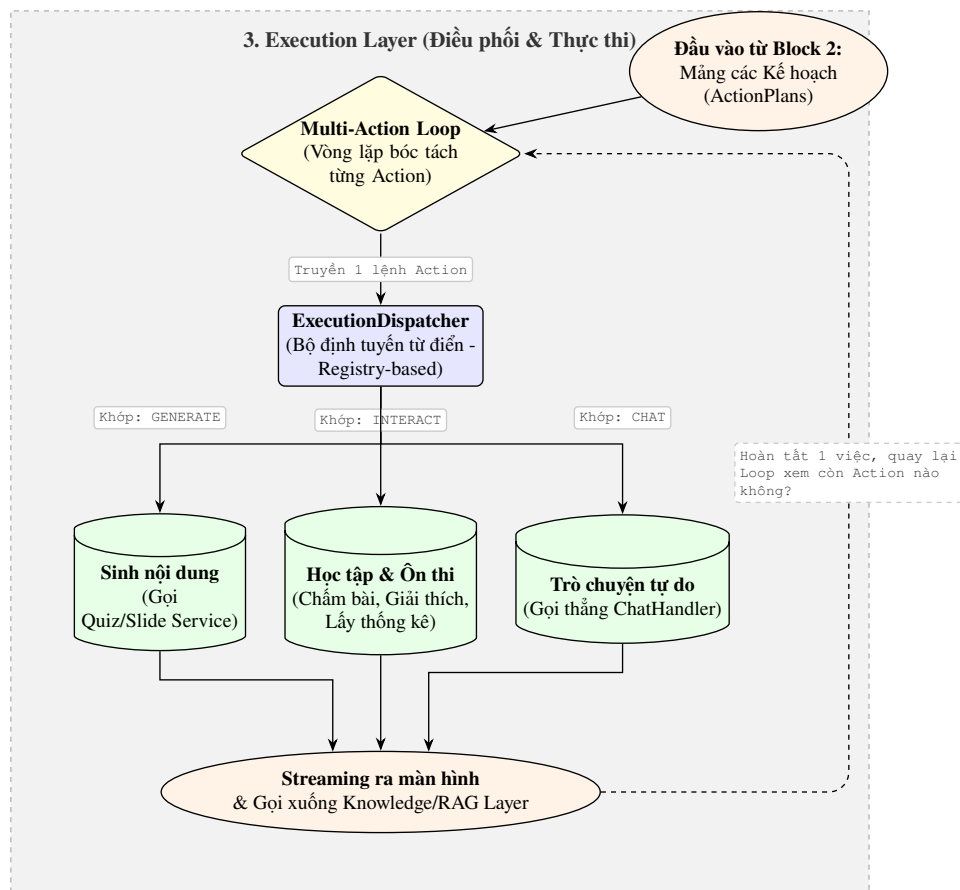
- **SessionManager:** Nhận truy vấn và xác định người dùng đang tiếp nối một chuỗi hội thoại cũ (cần tải lại ngữ cảnh quá khứ) hay khởi tạo tác vụ mới hoàn toàn. Khối này sẽ trực tiếp duy trì không gian *Session* phù hợp.

- **ActionPlanner:** Dựa trên cấu trúc Session nhận được từ SessionManager và các đặc trưng từ danh sách Intent, mô-đun này sẽ ánh xạ các ý định chung chung thành cấu trúc *ActionPlans* (Danh sách Lệnh hành động) rõ cấp bậc và không trùng lặp (deduplicated).

Đầu ra cuối cùng là danh sách tác vụ rành mạch, kết thúc vai trò nền tảng để đẩy sang Lớp Thực thi (Execution Layer).

0.3.3 Thiết kế Lớp Thực thi (Execution Layer)

Lớp Thực thi đóng vai trò trung tâm trong chuỗi xử lý đa nhiệm của hệ thống, có nhiệm vụ bóc tách danh sách các thao tác đã được Lập kế hoạch và phân phối chúng đến đúng bộ dịch vụ chuyên trách (Handler). Cơ chế định tuyến này giúp quy trình diễn ra liên tục, linh hoạt và độ chính xác cao.



Hình 0.4: Sơ đồ vòng lặp Đa tác vụ (Multi-Action Loop) tại Lớp Thực thi

Quá trình thực thi trong lớp này gồm các thành phần cốt lõi:

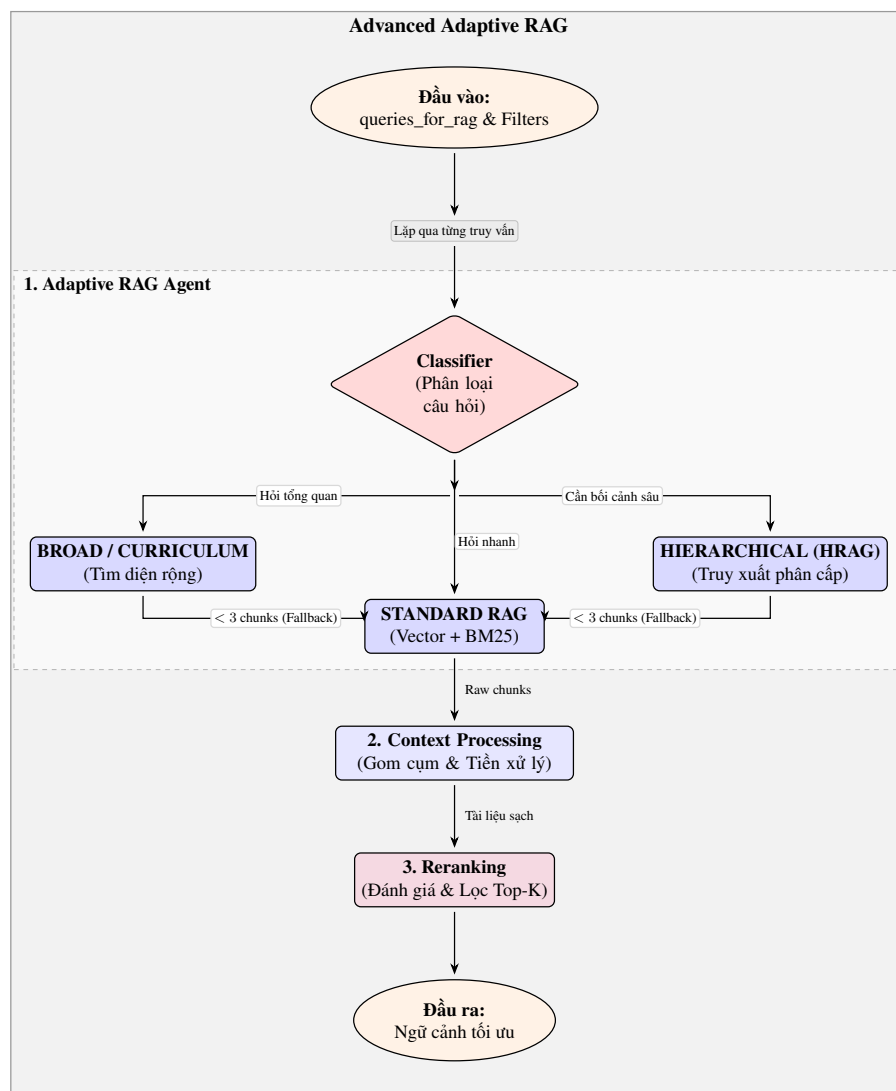
- **Multi-Action Loop:** Hệ thống sẽ duyệt qua mảng hành động (ActionPlans) theo phương thức lặp vòng. Vòng lặp đóng vai trò như một bộ đếm trạng thái, dừng lại khi tất cả Action đã được bóc tách hoàn chỉnh.
- **Execution Dispatcher:** Dựa vào khóa định danh của mỗi tác vụ, Dispatcher

sử dụng cơ chế nối kết từ điển (Registry-based) để chọn ra dịch vụ thích hợp gồm các module lớn như: Tạo Quiz/Slide, Xử lý Chấm điểm tương tác, hay Chat tự do (không cần RAG).

Kết thúc một luồng lệnh đơn lẻ, dữ liệu sẽ được trả về dạng Stream thời gian thực ra giao diện và gọi sâu xuống tầng Tri thức. Sau đó, tín hiệu sẽ vòng ngược lại Multi-Action Loop để tiếp tục gỡ lệnh kế tiếp.

0.3.4 Thiết kế Lớp Tri thức (Knowledge Layer)

Lớp Tri thức (Knowledge Layer) đóng vai trò nòng cốt trong việc giải quyết các tác vụ chuyên sâu (Knowledge-Intensive Tasks). Kiến trúc này được nâng cấp thành *Advanced Adaptive RAG*, không hoạt động trên một luồng cố định mà sử dụng Bộ phân loại (Classifier) để tự động định hướng cách thức truy xuất linh hoạt dựa trên dạng câu hỏi.



Hình 0.5: Cấu trúc xử lý và truy xuất thông tin của Knowledge Layer

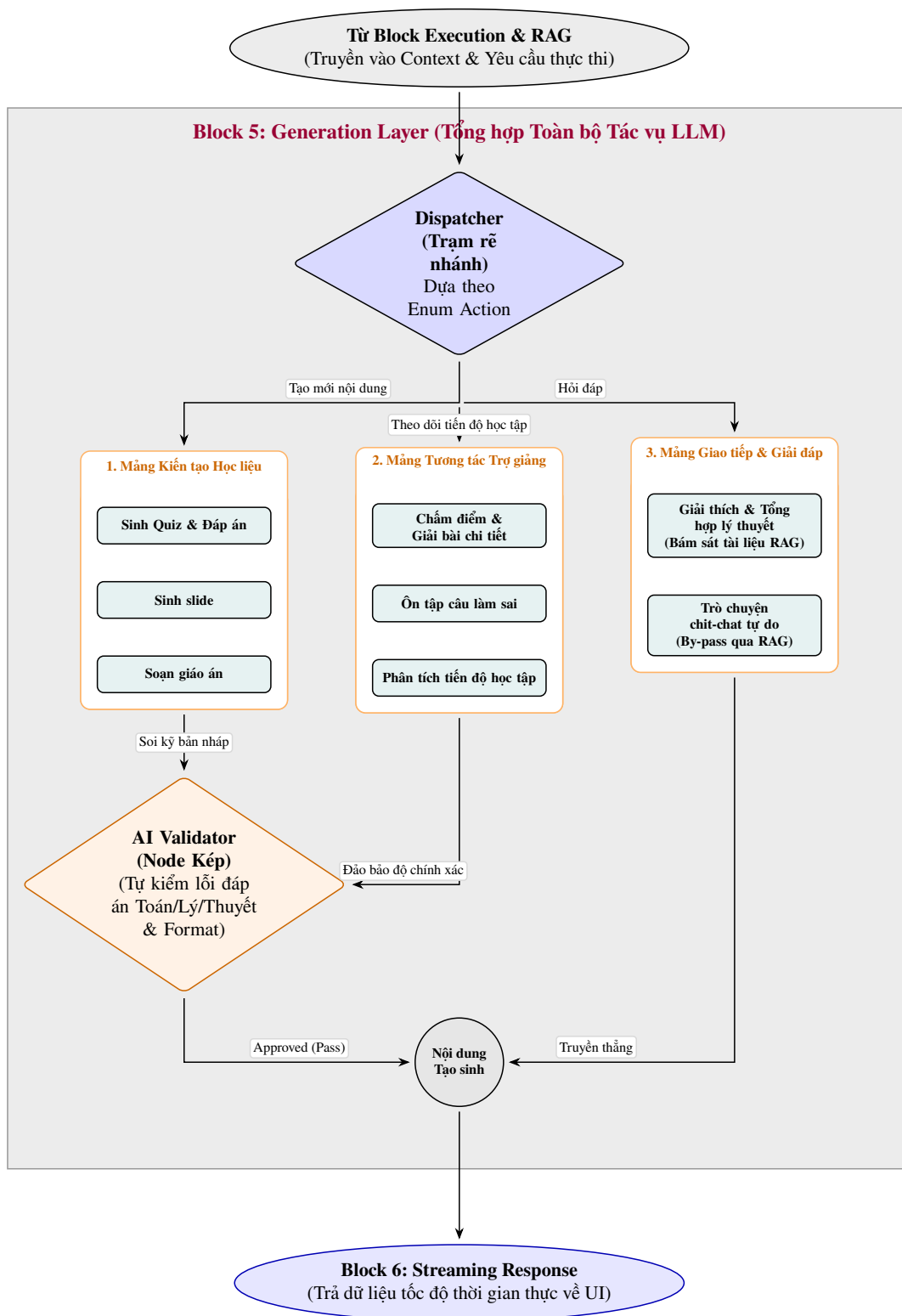
Đầu vào là mảng kết quả *queries* và các luồng lọc (filters) nhận được từ Lớp

trước đó. Quy trình chiết xuất thông tin tuần tự như sau:

- **1. Adaptive RAG Agent:** Bộ phân loại (Classifier) phân tích ý định sâu của truy vấn (hỏi nhanh, hỏi tổng quan hay cần bối cảnh sâu) để định tuyến sang dạng RAG tiêu chuẩn hoặc HRAG. Một thiết kế chốt chặn (Fallback Agent) sẽ tự động quay hồi về quy trình thông thường nếu việc rà soát trước đó rơi vào ngõ cụt (tìm được quá ít ngữ cảnh).
- **2. Context Processing:** Sau khi các Agent nhả ra đa dạng mảnh nguyên liệu (Raw chunks), quy trình sẽ gom tập trung và lọc đi các đoạn trùng lặp (Deduplication).
- **3. Reranking:** Cuối cùng, rổ tài liệu sau chuẩn hóa đi qua mô hình học máy (Cross-Encoder) để chấm điểm và tái xếp hạng. Top 5 Chunk giá trị nhất sẽ xuất ra làm thành phần ghép cốt lõi vào prompt để mô hình Generative phản hồi.

0.3.5 Thiết kế Lớp Sinh phản hồi và Hậu kiểm (Generation & Validation)

Cuối cùng, Response Generation nhận ngữ cảnh tinh lọc để biên soạn câu trả lời. Điểm khác biệt quan trọng là sự xuất hiện của Validator Agent, một lớp hậu kiểm phân luồng thực hiện đối soát câu trả lời vừa sinh ra với nguồn tri thức gốc để phát hiện các lỗi "hallucination", đảm bảo rằng mọi thông tin gửi tới người dùng đều đạt độ chính xác cao nhất (Hình 0.6).

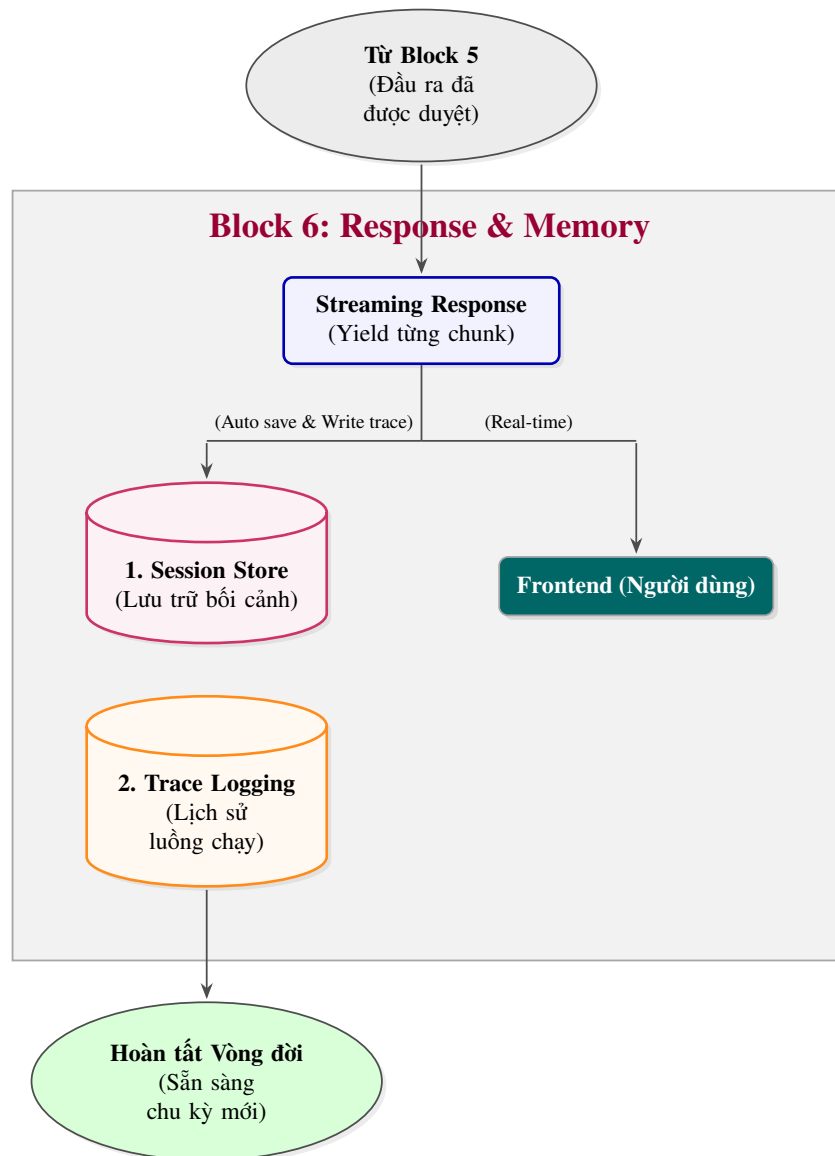


Hình 0.6: Giai đoạn phân luồng Tạo sinh và Hậu kiểm (Block 5)

0.3.6 Thiết kế Streaming Responsen, History Memory và Logging

Đây là chặng cuối cùng (Block 6) trong vòng đời xử lý của hệ thống, đóng vai trò như một cầu nối giao tiếp trực tiếp với người dùng và đồng thời dọn dẹp, lưu trữ trạng thái trước khi kết thúc phiên làm việc. Thiết kế của lớp này ưu tiên hai yếu tố: tốc độ hiển thị cho trải nghiệm người dùng (mượt mà thông qua Streaming) và

tính toàn vẹn dữ liệu của hệ thống (lưu vết và ngữ cảnh). Sơ đồ thao tác của Block 6 được thể hiện tại Hình 0.7.



Hình 0.7: Khối thao tác Phản hồi và Khắc ghi tiến trình (Block 6)

0.4 Đánh giá hiệu năng và Kết quả thực nghiệm

0.4.1 Phương pháp đánh giá

Hệ thống sử dụng phương pháp đánh giá định lượng (Quantitative Evaluation) dành riêng cho hệ thống Retrieval-Augmented Generation (RAG) thông qua Ragas framework. Tập dữ liệu kiểm thử (Testset) bao gồm 246 mẫu câu hỏi và đáp án tham chiếu (Reference) được trích xuất từ dữ liệu giáo khoa, tập trung vào nhiều cấp độ hành vi nhận thức từ Nhận biết, Thông hiểu đến Vận dụng.

Quá trình đánh giá được thực hiện trên 4 độ đo (Metrics) cốt lõi của Ragas:

- **Faithfulness (Độ trung thực):** Đo lường mức độ câu trả lời của mô hình bám

sát vào ngữ cảnh (context) mà hệ thống truy xuất được. Mức điểm cao chứng tỏ mô hình không bịa đặt thông tin (Hallucination).

- **Answer Relevancy (Độ phù hợp):** Đánh giá xem câu trả lời có giải quyết trực tiếp truy vấn ban đầu của người dùng hay không, tránh việc trả lời lan man, sai trọng tâm.
- **Context Precision (Độ chính xác ngữ cảnh):** Đánh giá khả năng xếp hạng của cụm truy xuất. Điểm số cao đồng nghĩa với việc các tài liệu liên quan nhất được xếp ở các thứ hạng đầu (Top-K).
- **Context Recall (Độ bao phủ ngữ cảnh):** Đo lường khả năng hệ thống tìm ra đầy đủ các thông tin cần thiết từ Cơ sở tri thức (Knowledge Base) so với đáp án tiêu chuẩn (Reference).

0.4.2 Kết quả thực nghiệm phân hệ RAG

Sau quá trình chạy tự động 246 mẫu thử trên toàn bộ pipeline của hệ thống, kết quả điểm số đánh giá trung bình trên các độ đo như sau:

Chỉ số đánh giá (Metric)	Điểm số trung bình (Mean)
Faithfulness (Độ trung thực)	0.9757
Answer Relevancy (Độ thiết thực của câu trả lời)	0.8571
Context Precision (Độ chính xác của ngữ cảnh)	0.8555
Context Recall (Độ bao phủ của ngữ cảnh)	0.9593

Bảng 1: Kết quả đánh giá hệ thống theo framework Ragas

Phân tích kết quả:

- **Faithfulness (0.9757)** và **Context Recall (0.9593)** đạt mức điểm rất cao (trên 0.95). Điều này minh chứng cho tính hiệu quả của chiến lược Hierarchical RAG kết hợp với cơ chế Chunking theo chuẩn phân cấp SGK. Hệ thống đã trích xuất được gần như hoàn hảo các đơn vị kiến thức cần thiết mà không xuất hiện tình trạng sinh thông tin ảo (Hallucination) từ phía Generative Model (Gemini). Validator Layer ở Block 5 cũng góp phần gia tăng chỉ số trung thực này.
- **Context Precision (0.8555)** và **Answer Relevancy (0.8571)** dao động ổn định ở mức 85%. Điểm số báo hiệu sự hiệu quả của mô hình tái xếp hạng Cross-Encoder trong việc đưa các tài liệu mang nhiều ý nghĩa lên đầu bảng, giúp Prompt được nhúng với ngữ cảnh làm trọng tâm, từ đó trả lời trúng đích câu hỏi người dùng.

0.4.3 Đánh giá độ trễ hệ thống (Latency)

Bên cạnh chất lượng phản hồi, hệ thống còn được đo đạc về mặt thời gian thực thi (Latency) nhằm đánh giá trải nghiệm người dùng thực tế. Các chỉ số thời gian được ghi nhận cho mỗi truy vấn đầy đủ như sau:

Thành phần thực thi	Thời gian trung bình (Giây)
Khối truy xuất (Retriever & Reranker)	4.579
Khối sinh câu trả lời (LLM Generator)	1.979
Tổng thời gian Pipeline	6.558

Bảng 2: Thống kê thời gian trễ của các khối kiến trúc trong hệ thống

Tổng thời gian cho một vòng lặp (từ lúc nhập truy vấn đến khi sinh mã vạch response trả về Frontend) đạt trung bình xấp xỉ 6.5 giây. Trọng số thời gian chủ yếu nằm ở khâu Retriever (hơn 4.5 giây) do hệ thống phải thực hiện Hybrid Search (bao gồm Dense Search kết hợp Custom BM25) sau đó cộng dồn với thời gian xử lý Reranker của Cross-Encoder để chất lọc dữ liệu. Tuy nhiên, việc áp dụng chiến lược sinh luồng trực tiếp (Streaming Response) ở Block 6 đã loại bỏ hiện tượng "đơ" giao diện, nâng cao trải nghiệm phản hồi (TTFT - Time to First Token được hạ thấp lý tưởng) và khiến người dùng cảm giác hệ thống trả lời tức thời.